Practical No.7

Develop a program to manage resource allocation for five processes (Google Drive, Firefox, Word Processor, Excel, and PowerPoint) using four types of resources (Printer, ROM, Hard Disk, and RAM). The program takes input for allocated, maximum, and available resources, calculates the current need of each process, and determines if a safe execution order exists using the Banker's Algorithm

Code:

```
#include <stdio.h>

#define P 5   // Number of processes
#define R 4   // Number of resources

int main() {
    int allocation[P][R], max[P][R], need[P][R];
    int available[R];
    int finish[P] = {0};
    int safeSeq[P];
    int work[R];
    int count = 0;

    char *processes[] = {
        "Google Drive",
        "Firefox",
        "Word Processor",
        "Excel",
        "PowerPoint"
    };

    char *resources[] = {
        "Printer",
        "ROM",
        "Hard Disk",
        "RAM"
    };

    printf("\nEnter Allocation Matrix:\n");
    for(int i = 0; i < P; i++) {
        printf("For %s:\n", processes[i]);
        for(int j = 0; j < R; j++) {
            printf("%s: ", resources[j]);
            scanf("%d", &allocation[i][j]);
        }
    }

    printf("\nEnter Maximum Matrix:\n");
    for(int i = 0; i < P; i++) {
        printf("For %s:\n", processes[i]);
        for(int j = 0; j < R; j++) {
            printf("%s: ", resources[j]);
            scanf("%d", &max[i][j]);
        }
    }
```

```
GNU nano 8.7
    }

    printf("\nEnter Available Resources:\n");
    for(int i = 0; i < R; i++) {
        printf("%s: ", resources[i]);
        scanf("%d", &available[i]);
        work[i] = available[i];
    }

    // Calculate Need Matrix
    for(int i = 0; i < P; i++) {
        for(int j = 0; j < R; j++) {
            need[i][j] = max[i][j] - allocation[i][j];
        }
    }

    printf("\nNeed Matrix:\n");
    for(int i = 0; i < P; i++) {
        printf("%s: ", processes[i]);
        for(int j = 0; j < R; j++) {
            printf("%d ", need[i][j]);
        }
        printf("\n");
    }

    // Banker's Algorithm
    while(count < P) {
        int found = 0;
        for(int i = 0; i < P; i++) {
            if(finish[i] == 0) {
                int flag = 1;
                for(int j = 0; j < R; j++) {
                    if(need[i][j] > work[j]) {
                        flag = 0;
                        break;
                    }
                }

                if(flag) {
                    for(int j = 0; j < R; j++)
                        work[j] += allocation[i][j];

                    safeSeq[count++] = i;
                    finish[i] = 1;
                    found = 1;
```

```
GNU nano 8.7                                                    Banker.c
        for(int j = 0; j < R; j++) {
            printf("%d ", need[i][j]);
        }
        printf("\n");
    }

    // Banker's Algorithm
    while(count < P) {
        int found = 0;
        for(int i = 0; i < P; i++) {
            if(finish[i] == 0) {
                int flag = 1;
                for(int j = 0; j < R; j++) {
                    if(need[i][j] > work[j]) {
                        flag = 0;
                        break;
                    }
                }

                if(flag) {
                    for(int j = 0; j < R; j++)
                        work[j] += allocation[i][j];

                    safeSeq[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }

        if(found == 0) {
            printf("\nSystem is NOT in safe state (Deadlock possible)\n");
            return 0;
        }
    }

    printf("\nSystem is in SAFE state.\nSafe sequence:\n");
    for(int i = 0; i < P; i++)
        printf("%s -> ", processes[safeSeq[i]]);

    printf("END\n");

    return 0;
}
```

Output:

```
Chanchal@DESKTOP-GARLJAI MSYS ~
# nano Banker.c

Chanchal@DESKTOP-GARLJAI MSYS ~
# gcc Banker.c -o Banker

Chanchal@DESKTOP-GARLJAI MSYS ~
# ./Banker

Enter Allocation Matrix:
For Google Drive:
Printer: 1
ROM: 2
Hard Disk: 3
RAM: 0
For Firefox:
Printer: 1
ROM: 2
Hard Disk: 4
RAM: 5
For Word Processor:
Printer: 2
ROM: 3
Hard Disk: 4
RAM: 0
For Excel:
Printer: 1
ROM: 4
Hard Disk: 2
RAM: 2
For PowerPoint:
Printer: 2
ROM: 4
Hard Disk: 2
RAM: 0

Enter Maximum Matrix:
For Google Drive:
Printer: 3
ROM: 2
Hard Disk: 1
RAM: 2
For Firefox:
Printer: 2
ROM: 3
Hard Disk: 1
RAM: 0
For Word Processor:
```

```
Hard Disk: 1
RAM: 0
For Word Processor:
Printer: 9
ROM: 3
Hard Disk: 2
RAM: 4
For Excel:
Printer: 1
ROM: 3
Hard Disk: 2
RAM: 5
For PowerPoint:
Printer: 2
ROM: 2
Hard Disk: 1
RAM: 0

Enter Available Resources:
Printer: 2
ROM: 3
Hard Disk: 1
RAM: 1

Need Matrix:
Google Drive: 2 0 -2 2
Firefox: 1 1 -3 -5
Word Processor: 7 0 -2 4
Excel: 0 -1 0 3
PowerPoint: 0 -2 -1 0

System is in SAFE state.
Safe sequence:
Firefox -> Excel -> PowerPoint -> Google Drive -> Word Processor -> END

Chanchal@DESKTOP-GARLJAI MSYS ~
# 
```